

```
#####  
# COVACIEL - SYSTÈME DE CHRONOMÉTRAGE MULTI-VOITURES  
# Réalisé par Antoine NICOLAS  
#####
```

```
import time  
import os  
import cv2  
import csv  
import threading  
import serial  
from datetime import datetime  
from pyzbar.pyzbar import decode
```

```
import board  
import busio  
from adafruit_bus_device.i2c_device import I2CDevice  
import adafruit_vl53l0x
```

```
#####  
# CONFIGURATION GÉNÉRALE  
#####
```

```
USB_CAMERA_INDEX = 0 # Index de la caméra USB
```

```
# Canaux du TCA9548A pour les capteurs VL53L0X  
CHANNEL_LIGNE_GAUCHE = 0  
CHANNEL_LIGNE_DROITE = 1  
CHANNEL_VERTICAUX = [3, 4, 5, 6, 7] # Capteurs verticaux (détection passage)
```

```
# Seuils de détection (distance en mm)  
SEUIL_LIGNE = 350  
SEUIL_VERTICAL = 455
```

```
DELAJ_LOOP = 0.001 # Délai très court pour la boucle principale
```

```
# XBee  
XBEE_PORT = "/dev/ttyUSB0"  
XBEE_BAUD = 9600
```

```
# Dossier d'archivage  
BASE_DIR = "/home/pi/capture_covaciel"
```

```
#####  
# INITIALISATION XBEE  
#####
```

```
def init_xbee():
    """
    Ouvre le port série du XBee.
    Retourne l'objet Serial ou None si erreur.
    """
    try:
        print(f"[XBEE] Initialisation sur {XBEE_PORT} @ {XBEE_BAUD}...")
        x = serial.Serial(XBEE_PORT, XBEE_BAUD, timeout=1, write_timeout=1)
        time.sleep(1) # Petit délai pour stabiliser
        print("[XBEE] OK")
        return x
    except Exception as e:
        print(f"[XBEE][ERREUR] Impossible d'ouvrir {XBEE_PORT} : {e}")
        return None

xbee = init_xbee()
```

```
def xbee_send_line(msg, repeat=5):
    """
    Envoie un message XBee plusieurs fois pour garantir la réception.
    """
    if xbee is None or not xbee.is_open:
        print(f"[XBEE][WARN] '{msg}' non envoyé (port fermé)")
        return

    data = (msg + "\n").encode("ascii") # Une seule fin de ligne

    print(f"[XBEE][INFO] Envoi '{msg}' x{repeat}")
    for i in range(repeat):
        try:
            xbee.write(data)
            xbee.flush()
            print(f"[XBEE] → {msg} ({i+1}/{repeat})")
            time.sleep(0.08) # Petit délai pour éviter collisions radio
        except Exception as e:
            print(f"[XBEE][ERREUR] {e}")
            break
```

```
#####
# THREAD CAMÉRA USB
#####
```

```
last_frame = None    # Dernière image capturée
camera_running = True # Flag pour arrêter proprement le thread
```

```

def camera_thread():
    """
    Thread indépendant qui capture en continu les images de la caméra USB.
    Avantage : la capture ne bloque jamais la boucle principale.
    """

    global last_frame, camera_running

    while camera_running:
        # Ouverture de la caméra
        cap = cv2.VideoCapture(USB_CAMERA_INDEX, cv2.CAP_V4L2)

        if not cap.isOpened():
            print("[CAM] Caméra USB non détectée, retry...")
            time.sleep(1)
            continue

        print("[CAM] Caméra USB OK")

        # Paramètres vidéo
        cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
        cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
        cap.set(cv2.CAP_PROP_FPS, 30)

        # Boucle de capture
        while camera_running:
            ret, frame = cap.read()
            if ret:
                last_frame = frame # On stocke la dernière image
            else:
                print("[CAM] Perte du flux USB, reconnexion...")
                break

        cap.release()

    # Lancement du thread caméra
    threading.Thread(target=camera_thread, daemon=True).start()
    time.sleep(1) # Laisse le temps à la caméra de démarrer

#####
# INITIALISATION I2C + TCA9548A
#####

# Bus I2C principal
i2c = busio.I2C(board.SCL, board.SDA, frequency=400000)

# Multiplexeur I2C TCA9548A (adresse 0x70)
tca = I2CDevice(i2c, 0x70)

```

```

def select_channel(ch):
    """
    Sélectionne un canal du TCA9548A.
    Indispensable pour isoler chaque VL53L0X.
    """
    with tca as d:
        d.write(bytes([1 << ch])) # Active uniquement le canal demandé
        time.sleep(0.002) # Délai court pour stabiliser

def init_capteur(ch, retries=3):
    """
    Initialise un capteur VL53L0X sur un canal donné.
    Réessaye plusieurs fois car ces capteurs sont sensibles au timing.
    """
    for attempt in range(retries):
        try:
            select_channel(ch)
            cap = adafruit_vl53l0x.VL53L0X(i2c)
            return cap
        except Exception:
            time.sleep(0.05)
    return None

#####
# DIAGNOSTIC DES CAPTEURS
#####

print("Initialisation des capteurs...")

capteur_ligne_gauche = init_capteur(CHANNEL_LIGNE_GAUCHE)
capteur_ligne_droite = init_capteur(CHANNEL_LIGNE_DROITE)

capteurs_verticaux = []
canaux_verticaux_actifs = []

# On teste chaque capteur vertical
for ch in CHANNEL_VERTICAUX:
    c = init_capteur(ch)
    if c:
        capteurs_verticaux.append(c)
        canaux_verticaux_actifs.append(ch)

print("\n===== DIAGNOSTIC CAPTEURS =====")

if capteur_ligne_gauche:
    print(f"[LIGNE] Gauche : OK (canal {CHANNEL_LIGNE_GAUCHE})")

```

```

else:
    print(f"[LIGNE] Gauche : ABSENT (canal {CHANNEL_LIGNE_GAUCHE})")

if capteur_ligne_droite:
    print(f"[LIGNE] Droite : OK (canal {CHANNEL_LIGNE_DROITE})")
else:
    print(f"[LIGNE] Droite : ABSENT (canal {CHANNEL_LIGNE_DROITE})")

print("\n[VERTICAUX]")
for ch in CHANNEL_VERTICAUX:
    if ch in canaux_verticaux_actifs:
        print(f" Canal {ch} : OK")
    else:
        print(f" Canal {ch} : ABSENT")
print("=====\n")

```

```

#####
# LECTURE QR CODE
#####

```

```

def lire_qr_robuste(image):
    """
    Lecture robuste du QR code :
    - recadrage au centre
    - conversion en niveaux de gris
    - plusieurs traitements (normal, OTSU, flou)
    """
    if image is None:
        return None

    h, w = image.shape[:2]

    # On recadre au centre pour éviter les bords inutiles
    crop = image[int(h*0.2):int(h*0.8), int(w*0.2):int(w*0.8)]
    gray = cv2.cvtColor(crop, cv2.COLOR_BGR2GRAY)

    # Plusieurs tentatives de décodage
    for img in [
        gray,
        cv2.threshold(gray, 0, 255, cv2.THRESH_OTSU)[1],
        cv2.GaussianBlur(gray, (5, 5), 0)
    ]:
        codes = decode(img)
        if codes:
            return codes[0].data.decode()

    return None

```

```
#####
# ARCHIVAGE DES RÉSULTATS
#####

date_course = datetime.now().strftime("%d-%m-%Y_%H-%M-%S")
course_dir = os.path.join(BASE_DIR, date_course)
os.makedirs(course_dir, exist_ok=True)

csv_path = os.path.join(course_dir, "resultats.csv")

# Création du CSV
with open(csv_path, "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["voiture_id", "tour", "temps_tour", "temps_total", "timestamp", "photo"])

#####
# DEMANDE DU NOMBRE DE TOURS
#####

def demander_nb_tours():
    """
    Demande utilisateur : combien de tours pour gagner ?
    """
    while True:
        try:
            n = int(input("Combien de tours ? "))
            if n > 0:
                return n
        except:
            pass
        print("Nombre invalide.")

nb_tours = demander_nb_tours()

#####
# ENVOI DU START AUX VOITURES
#####

print("[XBEE] Envoi START...")
xbee_send_line("START")
print("[XBEE] START envoyé.")

#####
```

```
# DÉTECTION DU SENS DE PASSAGE
```

```
#####
```

```
def detecter_sens():
```

```
    """
```

```
    Détecte si les voitures roulent vers la GAUCHE ou la DROITE.
```

```
    Basé sur les capteurs bas gauche/droite.
```

```
    """
```

```
    if not capteur_ligne_gauche or not capteur_ligne_droite:
```

```
        print("[SENS] Capteurs ligne absents → sens INCONNU")
```

```
        return "INCONNU"
```

```
    print("[SENS] Détection...")
```

```
    while True:
```

```
        # Lecture capteur gauche
```

```
        select_channel(CHANNEL_LIGNE_GAUCHE)
```

```
        g = capteur_ligne_gauche.range
```

```
        # Lecture capteur droit
```

```
        select_channel(CHANNEL_LIGNE_DROITE)
```

```
        d = capteur_ligne_droite.range
```

```
        # Logique de détection
```

```
        if g < SEUIL_LIGNE and d >= SEUIL_LIGNE:
```

```
            print("→ Sens GAUCHE")
```

```
            return "GAUCHE"
```

```
        if d < SEUIL_LIGNE and g >= SEUIL_LIGNE:
```

```
            print("→ Sens DROITE")
```

```
            return "DROITE"
```

```
        time.sleep(0.01)
```

```
sens = detecter_sens()
```

```
#####
```

```
# DÉTECTION MULTI-VOITURES (capteurs verticaux)
```

```
#####
```

```
def detecter_voitures_vertical():
```

```
    """
```

```
    Retourne la liste des indices des capteurs verticaux activés.
```

```
    Plusieurs voitures peuvent passer en même temps.
```

```
    """
```

```
    detections = []
```

```

for idx, cap in enumerate(capteurs_verticaux):
    ch = canaux_verticaux_actifs[idx]
    select_channel(ch)
    try:
        if cap.range < SEUIL_VERTICAL:
            detections.append(idx)
    except:
        pass
return detections

```

```

#####
# LOGIQUE DE COURSE
#####

```

```

depart = {}      # Temps du dernier passage
tours = {}       # Nombre de tours par voiture
temps_total = {} # Temps cumulé
voitures_finies = set()

```

```

# Anti-rebond : empêche les doubles scans
dernier_scan = {} # voiture_id → timestamp
COOLDOWN = 0.25   # 250 ms

```

```

print("=== CHRONO MULTI ===")
print(f"Sens détecté : {sens}")
print("En attente de voitures...")

```

```

#####
# BOUCLE PRINCIPALE
#####

```

```

try:
    while True:

        # Détection des voitures qui passent sous le portique
        idxs = detecter_voitures_vertical()

        if not idxs:
            time.sleep(DELAI_LOOP)
            continue

        for idx in idxs:
            ch = canaux_verticaux_actifs[idx]
            print(f"[PASSAGE] Capteur vertical canal {ch}")

            img = last_frame

```

```

if img is None:
    print("[CAM] Pas d'image")
    continue

# Lecture du QR code
voiture_id = lire_qr_robuste(img)
if voiture_id is None:
    print("[QR] Non détecté")
    continue

print(f"[QR] Voiture : {voiture_id}")

# ANTI-REBOND : empêche les doubles scans du même passage
now = time.time()
if voiture_id in dernier_scan:
    if now - dernier_scan[voiture_id] < COOLDOWN:
        print(f"[ANTI-REBOND] Ignoré (scan trop proche) pour {voiture_id}")
        continue
    dernier_scan[voiture_id] = now

# Premier passage → départ
if voiture_id not in depart:
    depart[voiture_id] = time.time()
    tours[voiture_id] = 0
    temps_total[voiture_id] = 0.0
    print(f"[DEPART] {voiture_id}")

# Passages suivants → tours
else:
    t_tour = time.time() - depart[voiture_id]
    depart[voiture_id] = time.time()
    tours[voiture_id] += 1
    temps_total[voiture_id] += t_tour

# Sauvegarde photo
photo = f"voiture_{voiture_id}_tour_{tours[voiture_id]}.jpg"
cv2.imwrite(os.path.join(course_dir, photo), img)

# Enregistrement CSV
with open(csv_path, "a", newline="") as f:
    writer = csv.writer(f)
    writer.writerow([
        voiture_id,
        tours[voiture_id],
        round(t_tour, 3),
        round(temps_total[voiture_id], 3),
        datetime.now().strftime("%H:%M:%S.%f")[:-3],
    ])

```

```

        photo
    ])

    print(f"[TOUR] {voiture_id} → {t_tour:.3f}s")

    # Vérification si la voiture a fini
    if tours[voiture_id] >= nb_tours:
        voitures_finies.add(voiture_id)
        print(f"[FIN] {voiture_id} a terminé")

# Si toutes les voitures détectées ont fini → fin de course
if len(voitures_finies) == len(tours) and len(tours) > 0:
    print("\n===== COURSE TERMINÉE =====")

    gagnant = min(voitures_finies, key=lambda v: temps_total[v])
    temps_gagnant = temps_total[gagnant]

    print(f"GAGNANT : Voiture {gagnant} avec {temps_gagnant:.3f} s")

    print("\n--- CLASSEMENT FINAL ---")
    classement = sorted(temps_total.items(), key=lambda x: x[1])

    for pos, (vid, ttot) in enumerate(classement, start=1):
        diff = ttot - temps_gagnant
        print(f"{pos}. Voiture {vid} → {ttot:.3f} s → +{diff:.3f} s")

    print("\n-----\n")

    print("[XBEE] Envoi END...")
    xbee_send_line("END")
    print("[XBEE] END envoyé.")

    break

time.sleep(DELAI_LOOP)

except KeyboardInterrupt:
    print("Arrêt manuel.")

camera_running = False
print("Fin du script.")

```